

PARECER CRITICO

Unidade II - Programacao Assistida por Inteligencia Artificial

Disciplina: Engenharia de Prompt e Aplicacoes em IA

Professora: Kadidja Valeria

Unidade: II - Programacao Assistida

Periodo de aulas: Abril de 2026

Nota: Este documento foi produzido com apoio de ferramenta de IA, conforme exigido pela proposta avaliativa. As fontes de pesquisa utilizadas estao referenciadas ao longo do texto e ao final do documento.

PARTE I - PARECER CRITICO DA UNIDADE II

Este parecer sintetiza as aprendizagens, praticas e reflexoes criticas desenvolvidas ao longo da Unidade II da disciplina de Engenharia de Prompt e Aplicacoes em IA, com foco na colaboracao Humano-IA no contexto da programacao assistida e da automacao de scripts.

1. Introducao e Contextualizacao

A Unidade II apresentou um novo paradigma no desenvolvimento de software: a programacao assistida por Inteligencia Artificial. Conforme fundamentado por Carraro (2023), a IA nao substitui o programador, mas amplifica suas capacidades cognitivas, permitindo que ele se concentre em decisoes de alto nivel. Esse conceito foi o fio condutor de todas as aulas do modulo.

Dados consolidados de instituicoes como McKinsey, PwC, MIT, Stanford, Harvard, FGV e IBGE confirmam o impacto direto do uso de IA na carreira profissional: maiores ganhos salariais, maior volume e qualidade de entrega, crescimento e promocao acelerados, e maior atratividade para recrutadores. Como destacado nos slides da Aula 24/04, a IA ja nao e o futuro da engenharia de software - ela e o padrao atual de produtividade.

A Unidade foi estruturada em quatro momentos complementares, cada um aprofundando um aspecto distinto da colaboracao Humano-IA: introducao ao pair programming com IA (Aula 07/04), sprint pratico de automacao (Aula 17/04), boas praticas e etica (Aula 24/04)

e aprofundamento teórico sobre os fundamentos da colaboração (material AI Pair Programming Revolution).

2. A Evolução: De Assistentes Reativos a Agentes Autônomos

Um dos conceitos mais marcantes da unidade foi a diferenciação entre assistentes reativos e agentes autônomos (Aula 17/04). Enquanto os assistentes reativos como o autocompletar clássico atuam de forma passiva, focando em sintaxe e trechos curtos de código, os agentes autônomos atuais planejam, executam, refatoram e depuram, atuando como parceiros intelectuais proativos focados na resolução de problemas complexos.

Historicamente, as ferramentas evoluíram de dicionários estáticos nos anos 2000 para ferramentas de análise estática nos anos 2010, até o salto dos LLMs e do GitHub Copilot em 2020-2021. A era atual (2023+) é marcada pela integração de modelos como GPT e Claude, capazes de dialogar sobre arquitetura e manter compreensão profunda de contexto.

Essa evolução ressignifica o papel do desenvolvedor: ele deixa de ser um 'digitador de sintaxe' para tornar-se o Tech Lead de um parceiro incansável, porém júnior - como metaforizado na Aula 07/04. A máquina sugere a velocidade; o humano garante a direção e a arquitetura.

3. Arsenal Técnico: Ferramentas em Foco

A Unidade II trabalhou com três ferramentas principais, cada uma com um perfil e superpoder específico:

GitHub Copilot	Co-piloto Tático	Sugestões inline e refatoração inteligente auxiliares e otimizações	
Replit + Ghostwriter	Base de Operações	Ambiente Cloud com execução imediata	Hospedar e testar scripts automatizados
Google Colab		Ambiente interativo com execução por células validar lógica de funcionamento	

4. Workflow Sustentável com IA

A Aula 24/04 consolidou o fluxo de trabalho sustentável em cinco fases cíclicas, cujo princípio central é: maestria não é conhecer a ferramenta - é integrar a IA sem comprometer a qualidade.

- Planejamento: O humano define arquitetura de alto nível, integrações complexas e

decisões de trade-off. A IA gera o código boilerplate e implementações padrão.

- Implementação: Desenvolvimento guiado por comentários (Murta, 2023). O humano escreve comentários descrevendo a intenção; a IA gera a implementação inicial; o humano valida e refina.
- Revisão Crítica: Checklist de QA obrigatório - o código faz exatamente o que foi pedido? Há edge cases não tratados? É eficiente? Segue padrões do projeto? Há vulnerabilidades óbvias?
- Teste: A IA é usada para 'quebrar' o próprio código gerado, gerando testes unitários abrangentes baseados em bugs comuns (Lee, Goldberg e Kohane, 2024).
- Manutenção: A IA atua como tradutora de código legado - explica linha a linha, analisa impacto de mudanças e gera testes de regressão antes de qualquer refatoração.

5. Prática: O Sprint de Automação (Aula 17/04)

A aula-sprint de 17/04 foi o momento mais prático da unidade, estruturada em três missões progressivas que aplicaram os conceitos teóricos em desafios reais:

Missão 01 - Automação de Arquivos

Construção de um organizador automático de arquivos em Python utilizando as bibliotecas `os` e `shutil`. O script cria pastas por extensão de arquivo (.pdf, .jpg, etc.), move os arquivos corretamente e executa sem erros de permissão. O Ghostwriter no Replit foi utilizado para sugerir as funções de manipulação de diretórios.

Missão 02 - Consulta a APIs Públicas

Desenvolvimento de um script que consulta a API ViaCEP e retorna informações de endereço formatadas. A estratégia didática foi provocar um erro intencional na URL e usar a IA do Replit para identificar a falha e sugerir tratamento de exceções (try/except). O resultado demonstrou na prática como a IA acelera a depuração.

Missão 03 - Sistema de Notificações

Criação de um programa autônomo que dispara alertas quando uma condição crítica é atingida. O GitHub Copilot foi utilizado para gerar o corpo das funções auxiliares a partir do nome da função, e em seguida refatorar o código para torná-lo mais modular e limpo.

A retrospectiva do sprint revelou claramente os limites naturais da colaboração: a IA brilhou na geração de boilerplate, sugestões de sintaxe e formatação JSON, mas a intervenção humana foi crítica para garantir a lógica de negócio, tratar edge cases e tomar decisões de segurança e arquitetura.

6. Ética, Segurança e Responsabilidade Legal

A dimensão ética foi apresentada de forma direta na Aula 24/04, com base em Barcaui

(2025): assistentes de IA não tem consciência de segurança. A responsabilidade é integralmente humana.

Tres riscos principais foram identificados:

- Vulnerabilidades de Segurança: O treinamento em dados da internet aberta expõe o código gerado a riscos de Injeção de SQL, XSS e uso de bibliotecas desatualizadas. Ação recomendada: uso de ferramentas SAST/DAST e sanitização de dados de entrada.
- Propriedade Intelectual: Risco de violação de licenças, especialmente código GPL em produtos fechados. Ação recomendada: estabelecer políticas de auditoria (Mello, 2024).
- Vies e Discriminação: A replicação de preconceitos históricos em algoritmos. Ação recomendada: revisão crítica das operações lógicas buscando justiça e equidade.

Quanto a responsabilidade legal, Lee, Goldberg e Kohane (2024) são enfáticos: ferramentas de IA são assistentes, não agentes autônomos. O desenvolvedor que aceita e implementa código gerado por IA assume total responsabilidade por esse código. Em sistemas críticos - aprovação de crédito, diagnóstico médico, triagem de RH - a explicabilidade não é um luxo, é uma exigência (Cordova, 2025).

7. Os 5 Princípios Fundamentais da Colaboração Humano-IA

A Aula 24/04 sintetizou cinco princípios que devem reger toda colaboração responsável com IA no desenvolvimento de software:

1	Transparencia	Se a IA gerar algo que você não compreende totalmente, pause e aprenda antes de aceitar.
2	Verificação	Nunca aceite código gerado sem validar o comportamento prático com testes reais.
3	Progressão	Use com confiança onde já tem domínio; use com extrema cautela ao aprender conceitos novos.
4	Documentação	Explique nos comentários o porque daquela abordagem ter sido escolhida.
5	Revisão por Pares	Trate o código da IA com o mesmo rigor (ou maior) que o código humano da equipe.

8. Síntese Crítica: O Desenvolvedor Aumentado

A Unidade II consolidou uma visão clara sobre o papel do profissional de tecnologia na era da IA. A fórmula apresentada é precisa: intuição humana mais velocidade da IA resulta no desenvolvedor aumentado.

O valor do desenvolvedor não está mais em digitar sintaxe com rapidez, mas em arquitetar soluções lógicas e seguras. A confiança na IA deve ser calibrada de acordo com a complexidade e o risco: alta delegação para estruturas iniciais e boilerplate; supervisão total para autenticação, algoritmos de segurança e decisões arquiteturais críticas (Barcaui, 2025; Eysenck e Eysenck, 2023).

Como afirmado no material da Aula 07/04: a IA não substitui o desenvolvedor, mas amplia sua capacidade criativa e produtiva. O papel humano é decidir, validar e dar direção. Dominar essas práticas é vital para a sobrevivência na carreira e a escalabilidade da produtividade.

REFERENCIAS BIBLIOGRAFICAS

BARCAUI, A. Gestão ágil de projetos com Inteligência Artificial. Rio de Janeiro: FGV Editora, 2025.

CARRARO, T. Programação Assistida por IA: Paradigmas e Práticas. São Paulo: Novatec, 2023.

CORDOVA, R. Transparência e Explicabilidade em Sistemas de IA. Lisboa: IST Press, 2025.

EYSENCK, M. W.; EYSENCK, M. Fundamentos de psicologia cognitiva. 3. ed. Porto Alegre: Artmed, 2023.

LEE, P.; GOLDBERG, C.; KOHANE, I. The AI Revolution in Medicine: GPT-4 and Beyond. Boston: Pearson, 2024.

MELLO, R. Propriedade Intelectual e IA Generativa: desafios jurídicos contemporâneos. Brasília: OAB Editora, 2024.

MURTA, L. Engenharia de Software Moderna. Belo Horizonte: UFMG, 2023. Disponível em: <https://engsoftmoderna.info>

Parecer Individual - Unidade II

Disciplina: Engenharia de Prompt e Aplicações em IA | Professora: Kadidja Valéria |
Prazo: 29/04/2026

A. Identificação da Atividade

Tipo de atividade	Parecer Individual - Atividade Avaliativa (A2)
Disciplina	Engenharia de Prompt e Aplicações em IA

Unidade avaliada	Unidade II - Programacao Assistida
Professora	Kadidja Valeria Oliveira
E-mail	kadidja.oliveira@udf.edu.br
Entrega	Repositorio GitHub da Disciplina + Link enviado em 24/04
Prazo final	29 de abril de 2026

B. Descricao da Atividade

Conforme orientado na Aula 24/04, a atividade avaliativa da Unidade II consiste na elaboracao de um parecer individual abordando:

- As praticas desenvolvidas ao longo da unidade: sugestoes de codigo, refatoracao assistida por IA e automacao de scripts Python.
- A relacao com o conteudo estudado: workflow sustentavel com IA, principios eticos, ferramentas utilizadas (Copilot, Replit/Ghostwriter).
- Transparencia metodologica: indicacao das fontes de pesquisa utilizadas e qual ferramenta de IA apoiou a producao do texto.
- Base obrigatoria: escuta do Podcast da Unidade 2 disponibilizado no Blackboard (Fundamentacao Teorica Conceitual).

C. Resposta Desenvolvida: O Que Foi Praticado

C.1 Ferramentas e Praticas Utilizadas

Ao longo da Unidade II, as seguintes ferramentas foram utilizadas diretamente nas atividades praticas:

- GitHub Copilot (VS Code): utilizado na Missao 03 para gerar o corpo de funcoes auxiliares do sistema de notificacoes a partir do nome da funcao, e em seguida para refatorar o codigo, tornando-o mais modular. O Copilot demonstrou eficiencia especialmente na geracao de estruturas boilerplate e na sugestao de padroes de nomenclatura.
- Replit + Ghostwriter: utilizado nas Missoes 01 e 02. No organizador de arquivos, o Ghostwriter sugeriu o uso correto das funcoes `shutil.move` e `os.makedirs` com `exist_ok=True`, evitando erros de permissao. Na consulta a API ViaCEP, foi utilizado para identificar o erro de requisicao provocado intencionalmente e sugerir a estrutura de tratamento de excecoes com `try/except`.
- Claude (Anthropic): utilizado como apoio na producao deste parecer e na organizacao das reflexoes criticas, com base nos materiais de aula fornecidos. Conforme exigido pela atividade, esta ferramenta e declarada explicitamente.

C.2 Relacao com o Conteudo Estudado

As praticas desenvolvidas ao longo da unidade confirmaram na pratica o que foi

apresentado teoricamente. O loop de colaboracao Humano-IA - instrucao, processamento, revisao e ajuste - foi vivenciado em cada missao do sprint (Aula 17/04). Nunca foi possivel simplesmente aceitar o codigo gerado sem leitura e validacao; a revisao critica foi sempre necessaria.

O cheat sheet de prompts da Aula 07/04 foi diretamente aplicado: a diferenca entre um prompt fraco ('Organize esses arquivos') e um prompt estruturado ('Crie uma funcao Python que organize arquivos de um diretorio em subpastas por extensao, usando shutil, criando as pastas caso nao existam') resultou em codigo completamente diferente em qualidade e funcionalidade.

A confianca calibrada (Barcaui, 2025) foi o conceito mais desafiador de aplicar: saber quando delegar completamente a IA (boilerplate, formatacao), quando supervisionar de perto (logica condicional) e quando assumir total controle (decisoes de seguranca) exige maturidade tecnica que se desenvolve progressivamente.

C.3 Limites e Forcas Observados

De acordo com a retrospectiva da Aula 17/04, os limites e forcas observados durante as praticas foram:

Geracao de estruturas boilerplate (repetitivas)	Garantir a logica de negocio (A regra faz sentido?)
Sugestoes de sintaxe esquecida	Tratamento de edge cases (entradas invalidas, CEPs inexistentes)
Formatacao rapida de dados JSON	Seguranca e decisoes arquiteturais
Identificacao de bugs de sintaxe em tempo real	Validacao da logica de fluxo do sistema de notificacoes

C.4 Reflexao Final: Ate Que Ponto Confiar na IA?

O debate proposto na Aula 24/04 - 'Ate que ponto podemos (e devemos) confiar na IA como verdadeira parceira de codigo?' - nao tem resposta binaria. A resposta mais precisa e: depende do contexto, da complexidade e do risco envolvido.

A adocao do Copilot em equipes de desenvolvimento, analisada no estudo de caso, revela uma balanca real: de um lado, ganho drastico de velocidade, apoio educacional para desenvolvedores juniores e refatoracao inteligente. Do outro, risco de dependencia cognitiva ('esquecer como programar'), problemas de licenciamento e introducao de codigo inseguro.

A sintese mais honesta e a formulada pela propria unidade: a IA e uma parceira, nao uma substituta. O valor do desenvolvedor migra de 'digitar sintaxe' para 'arquitetar solucoes logicas e seguras'. Quem dominar essa parceria tera vantagem competitiva real; quem depender dela sem senso critico assumira riscos que podem comprometer projetos e

carreiras.

DECLARACAO DE TRANSPARENCIA

Conforme exigido pela atividade, declaro que este parecer foi produzido com apoio da ferramenta Claude (Anthropic), utilizada para organizar e estruturar as reflexoes com base nos materiais de aula fornecidos (PDFs das Aulas 07/04, 17/04, 24/04 e AI Pair Programming Revolution). As ideias, argumentos e conclusoes refletem a compreensao critica do conteudo estudado. As referencias bibliograficas utilizadas foram extraidas dos proprios slides da disciplina.